

GROUP 1: Auth — APIs 3, 4, 5, 6

CONVENTIONS

- Response: `{success:bool, data?, message?}`
- Auth: Bearer JWT → `req.user = {user_id, roles:string[]}`
- SQL Server: `@param`
- Comment code: tối thiểu
- `token_hash` = `crypto.createHash('sha256').update(token).digest('hex')`

YÊU CẦU

Đọc code API 1 (Login) để hiểu cách tạo token, hash, response format → viết tiếp 4 API dưới đây theo đúng pattern.

DB schema liên quan:

```
refresh_tokens: refresh_token_id, user_id(FK), token_hash(UNIQUE),
device_info,
                ip_address, last_used_at, expires_at, revoked_at,
revocation_reason,
                replaced_by_token_id

password_reset_requests: reset_request_id, user_id(FK), destination_masked,
                        otp_hash, expires_at, verified_at, used_at,
                        attempt_count(def:0), max_attempts(def:5),

status(PENDING|VERIFIED|USED|EXPIRED|LOCKED|CANCELLED)
```

3 • Refresh Token `POST /api/v1/auth/refresh-token` | Public

Body: `{refresh_token}`

Steps:

1. Validate body không rỗng → 400
2. Tính hash: `token_hash = sha256(refresh_token)`
3. Query:

sql

```
SELECT rt.*, u.user_id, u.status AS user_status
FROM refresh_tokens rt
JOIN users u ON rt.user_id=u.user_id
WHERE rt.token_hash=@hash
```

4. Không tìm thấy → **401** "Token không hợp lệ"
5. `revoked_at IS NOT NULL` → **401** "Token đã bị thu hồi"
6. `expires_at < NOW()` → **401** "Token đã hết hạn"
7. `user.status != 'ACTIVE'` → **403** "Tài khoản bị khóa"
8. Lấy roles:

sql

```
SELECT r.role_code FROM user_roles ur JOIN roles r ON ur.role_id=r.role_id
WHERE ur.user_id=@uid AND ur.status='ACTIVE'
```

9. Tạo `newAccessToken` (JWT, same payload as Login)
10. **Token Rotation** — tạo refresh token mới:
 - `newRefreshToken = crypto.randomBytes(64).toString('hex')`
 - INSERT refresh_tokens (user_id, token_hash=sha256(newRT), expires_at=+30 days)
 - UPDATE cũ: `SET revoked_at=NOW(), revocation_reason='ROTATED', replaced_by_token_id=newId`
 - UPDATE: `SET last_used_at=NOW()` trên token cũ
11. Response **200**:

json

```
{
  "success": true,
  "data": {
    "access_token": "eyJ...",
    "refresh_token": "newToken...",
    "expires_in": 900
  }
}
```

4 • Logout (POST /api/v1/auth/logout) | Tất cả (Bearer required)

Body: {refresh_token}

Steps:

1. Lấy @uid từ JWT middleware
2. Validate body → 400 nếu thiếu refresh_token
3. Tính hash: sha256(refresh_token)
4. Revoke token:

sql

```
UPDATE refresh_tokens
SET revoked_at=SYSUTCDATETIME(), revocation_reason='LOGOUT', updated_at=SYSUTCDATETIME()
WHERE token_hash=@hash AND user_id=@uid AND revoked_at IS NULL
```

5. Không cần báo lỗi nếu token không tồn tại (idempotent logout)

→ 200: {success:true, message:"Đăng xuất thành công"}

5 • Forgot Password – Send OTP (POST /api/v1/auth/forgot-password) | Public

Body: {email?} hoặc {phone?} (1 trong 2)

Steps:

1. Validate: phải có email hoặc phone → 400
2. Tìm user:

sql

```
SELECT user_id, email, phone, status FROM users
WHERE (email=@email OR phone=@phone) AND deleted_at IS NULL
```

3. Nếu không tìm thấy → 200 (không tiết lộ email có tồn tại không, bảo mật)

{success:true, message:"Nếu tài khoản tồn tại, OTP đã được gửi"}

4. Nếu (status='SUSPENDED') → 403 "Tài khoản bị khóa"

5. Check rate limit: (SELECT COUNT(*) FROM password_reset_requests WHERE user_id=@uid AND status='PENDING' AND created_at > DATEADD(minute,-5,NOW())) → 429 nếu đã có request gần đây

6. Tạo OTP: `otp = String(Math.floor(100000 + Math.random() * 900000))` (6 chữ số)

7. Hash OTP: `otp_hash = bcrypt.hashSync(otp, 10)` HOẶC `sha256(otp)`

8. Mask destination:

- email: `a***@gmail.com` (giữ ký tự đầu + domain)
- phone: `090***7890` (giữ 3 đầu + 2 cuối)

9. Hủy request cũ còn PENDING:

sql

```
UPDATE password_reset_requests SET status='CANCELLED', updated_at=NOW()
WHERE user_id=@uid AND status='PENDING'
```

10. INSERT password_reset_requests:

sql

```
INSERT INTO password_reset_requests
(user_id, destination_masked, otp_hash, expires_at, status)
VALUES (@uid, @masked, @otp_hash, DATEADD(minute,5,NOW()), 'PENDING')
```

11. **DEMO MODE:** `console.log('OTP cho', destination, ':', otp)` (không gửi email thật)

12. Response 200:

json

```
{
  "success": true,
  "message": "Nếu tài khoản tồn tại, OTP đã được gửi",
  "data": {
    "destination_masked": "a***@gmail.com",
    "expires_in_seconds": 300
  },
  "_dev_otp": "123456"
}
```

```
> `_dev_otp` chỉ trả trong môi trường development (check
`process.env.NODE_ENV !== 'production'`)
```

6 · Reset Password – Confirm OTP (POST /api/v1/auth/reset-password) | Public

Body: {email?, phone?, otp, new_password}

Steps:

1. Validate: (email hoặc phone) + otp + new_password → **400**
2. Validate: (new_password) tối thiểu 6 ký tự → **400**
3. Tìm user: như API 5 → **400** "Tài khoản không tồn tại" (khác API 5 vì đây là action)
4. Lấy request mới nhất còn hiệu lực:

sql

```
SELECT TOP 1 * FROM password_reset_requests
WHERE user_id=@uid AND status IN ('PENDING','VERIFIED')
AND expires_at > SYSUTCDATETIME()
ORDER BY created_at DESC
```

5. Không tìm thấy → **400** "OTP không tồn tại hoặc đã hết hạn"
6. Check (attempt_count >= max_attempts) OR (status='LOCKED') → **429** "Đã vượt quá số lần thử"
7. Verify OTP: (bcrypt.compare(otp, request.otp_hash)) (hoặc (sha256(otp) === otp_hash))
8. Sai OTP:

sql

```
UPDATE password_reset_requests
SET attempt_count=attempt_count+1,
    status = CASE WHEN attempt_count+1 >= max_attempts THEN 'LOCKED' ELSE st
    updated_at=NOW()
WHERE reset_request_id=@rid
```

→ **400** "OTP không đúng" (còn attempt) hoặc **429** "Tài khoản tạm khóa reset" (hết attempt)

9. OTP đúng: a. (UPDATE password_reset_requests SET status='USED', verified_at=NOW(), used_at=NOW()) b. Hash mật khẩu mới: (bcrypt.hash(new_password, 10)) c. (UPDATE users SET password_hash=@newHash, updated_at=NOW()) d. Thu hồi tất cả refresh token:

sql

```
UPDATE refresh_tokens SET revoked_at=NOW(), revocation_reason='PASSWORD_RESET'
WHERE user_id=@uid AND revoked_at IS NULL
```

10. Response **200**: `{success:true, message:"Đổi mật khẩu thành công. Vui lòng đăng nhập lại."}`

CHECKLIST

- ☐ Route đăng ký vào index router (không cần auth middleware vì đều là Public, trừ API 4)
- ☐ API 4: cần Bearer JWT middleware
- ☐ API 3: token rotation — revoke token cũ, tạo token mới
- ☐ API 5: KHÔNG tiết lộ email/phone có tồn tại không (trả 200 dù không có)
- ☐ API 5: `_dev_otp` chỉ trả trong `NODE_ENV=development`
- ☐ API 6: tăng `attempt_count` khi sai OTP; `LOCKED` khi `>= max_attempts`
- ☐ API 6: thu hồi tất cả `refresh_tokens` sau reset thành công
- ☐ Không log hoặc trả OTP thật trong production